

ОТЧЕТ

по проекту собственной СУБД

Команда «Темщики»

Группа: ИУ5-23Б

Участники проекта:

- Тютюнник Николай
- Швечков Тимур

Введение

В современном программировании базы данных используются практически во всех информационных системах: интернет-магазинах, социальных сетях, банковских приложениях, системах учёта и других сервисах. Для управления данными используются системы управления базами данных (СУБД).

Целью данной работы является разработка собственной реляционной СУБД на языке программирования C++ с поддержкой базовых SQL-запросов и постоянного хранения данных.

Цель работы

Разработать собственную реляционную базу данных и СУБД к ней с поддержкой:

- хранения данных между запусками программы;
- базовых SQL-запросов;
- консольного интерфейса;
- автоматической сборки и запуска проекта.

Задачи проекта

В ходе выполнения проекта были поставлены следующие задачи:

- реализовать файловое хранение данных;
- реализовать создание и удаление баз данных;
- реализовать создание и удаление таблиц;
- реализовать операции добавления, изменения, удаления и выборки данных;
- создать библиотеку для взаимодействия с СУБД;
- реализовать консольный интерфейс;
- создать скрипты автоматической сборки и запуска проекта;
- применить паттерн проектирования.

Описание проекта

Разработанный проект представляет собой упрощённую реляционную СУБД, написанную на языке C++.

Система позволяет выполнять базовые SQL-запросы:

- CREATE DATABASE
- DROP DATABASE
- CREATE TABLE
- DROP TABLE
- INSERT
- SELECT
- UPDATE
- DELETE

Взаимодействие с системой осуществляется через консольный интерфейс.

Данные сохраняются в файлах и остаются доступными после повторного запуска программы.

Архитектура проекта

Компоненты проекта:

- src — исходный код проекта;
- build — собранные файлы;
- data — хранение данных;
- build.sh — автоматическая сборка;
- run.sh — запуск СУБД;
- CMakeLists.txt — настройка сборки.

Постоянное хранение данных

Для хранения информации используется файловая система.

Каждая таблица сохраняется в отдельном файле внутри директории data.

После завершения работы программы данные не удаляются и доступны после повторного запуска СУБД.

Библиотека для взаимодействия с СУБД

Взаимодействие клиента с системой реализовано через отдельную C++ библиотеку libsimpledb.a.

Клиентский код взаимодействует с СУБД только через API библиотеки без прямого доступа к файлам хранения.

Использованный паттерн проектирования

В проекте используется паттерн Singleton.

Паттерн применяется в классе DatabaseManager.

Singleton позволяет создать только один экземпляр менеджера базы данных во время работы программы. Это обеспечивает централизованное управление всеми операциями СУБД.

Поддерживаемые SQL-запросы

DDL-запросы:

Создание базы данных:

```
CREATE DATABASE test;
```

Удаление базы данных:

```
DROP DATABASE test;
```

Создание таблицы:

```
CREATE TABLE users (id INT, name TEXT);
```

Удаление таблицы:

```
DROP TABLE users;
```

DML-запросы:

Добавление данных:

```
INSERT INTO users VALUES (1, 'John');
```

Выборка данных:

```
SELECT * FROM users;
```

Изменение данных:

```
UPDATE users SET name='Bob' WHERE id=1;
```

Удаление данных:

```
DELETE FROM users WHERE id=1;
```

Интерфейс пользователя

Система использует консольный интерфейс.

После запуска пользователь вводит SQL-запросы вручную через терминал.

Пример запуска:

```
./run.sh 127.0.0.1 8080
```

После запуска появляется строка ввода:

```
db>
```

UML-диаграмма классов

DatabaseManager

- createDatabase()
- createTable()
- insert()
- select()

Table

- columns
- rows

Row

- values

Результаты работы

В ходе выполнения проекта была разработана собственная реляционная СУБД с поддержкой базовых SQL-запросов и постоянного хранения данных.

Были реализованы:

- файловое хранение;
- библиотека взаимодействия с БД;
- поддержка DDL и DML операций;
- консольный интерфейс;
- автоматическая сборка и запуск проекта.

Проект соответствует требованиям MVP.

Заключение

В результате выполнения работы была успешно разработана собственная система управления базами данных на языке C++.

Разработанная СУБД поддерживает основные SQL-запросы, обеспечивает хранение данных между запусками программы и имеет консольный интерфейс для взаимодействия с пользователем.

Проект может быть расширен дополнительной функциональностью, такой как полноценный SQL-парсер, система ролей и прав доступа, поддержка сетевого взаимодействия и графический интерфейс.